SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

ASSIGNMENT 2 – C++ PROGRAMMING

CO2 – Functions and Object-Oriented Programming

NAME: DENISHA SHREE R

REG NO: 192524016

Q1 – Call by Value, by Reference and by Pointer

Problem: A hostel office applies fee concessions. Implement the operation using pass-by-value, pass-by-reference and pass-by-pointer, predict the caller's fee, execute, and explain the difference.

Answer / Explanation

- Pass by value creates a separate parameter. Changing it changes only the local copy, so the caller's fee is unchanged.
- Pass by reference makes the parameter an alias for the caller's variable. The function changes the original fee.
- Pass by pointer receives the address of the caller's variable. Dereferencing the pointer changes the original fee.
- A const reference is useful when a function must read a large object without copying it and must not modify it.

Prediction

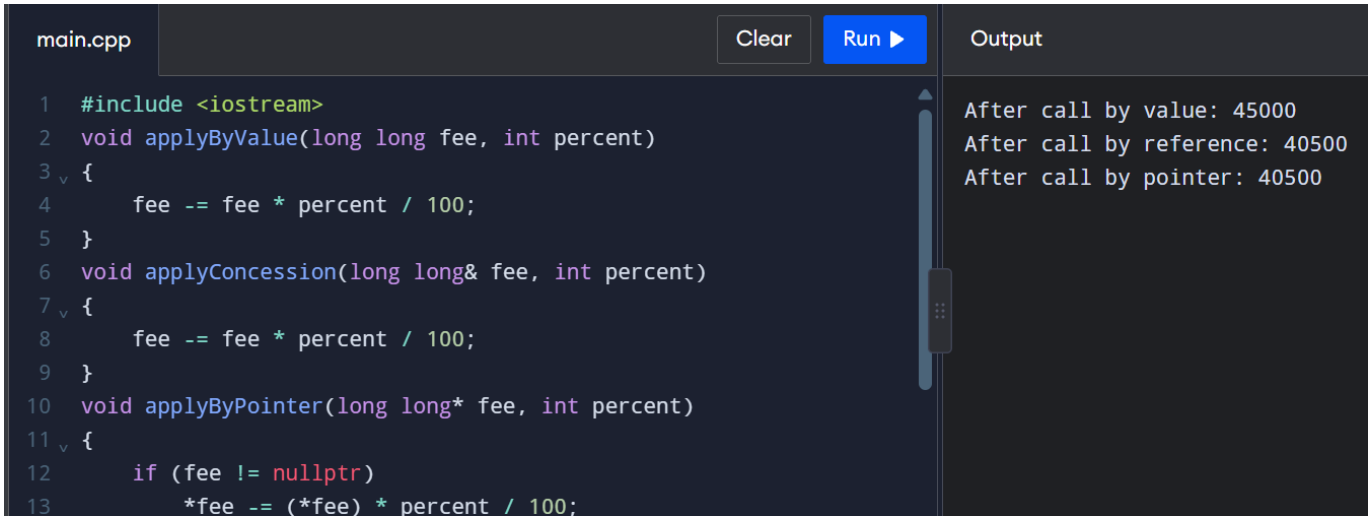
For an initial fee of 45,000 and a 10% concession: pass-by-value leaves the caller at 45,000; pass-by-reference changes it to 40,500; pass-by-pointer also changes it to 40,500.

Code

```
#include <iostream>
void applyByValue(long long fee, int percent)
{
   fee -= fee * percent / 100;
}
void applyConcession(long long& fee, int percent)
{
   fee -= fee * percent / 100;
}
void applyByPointer(long long* fee, int percent)
{
   if (fee != nullptr)
      *fee -= (*fee) * percent / 100;
}
int main()
```

```
{
    long long fee1 = 45000;
    long long fee2 = 45000;
    long long fee3 = 45000;
    applyByValue(fee1, 10);
    applyConcession(fee2, 10);
    applyByPointer(&fee3, 10);
    std::cout << "After call by value: " << fee1 << '\n';
    std::cout << "After call by reference: " << fee2 << '\n';
    std::cout << "After call by pointer: " << fee3 << '\n';
    return 0;
}
```

Expected output:



Alternative Concept and Complexity

A return-value design can use long long applyConcession(long long fee, int percent) and assign fee = applyConcession(fee, 10). All versions take O(1) time. A long long passed by value copies only a fixed-size scalar. For a large Student object, passing by value copies the whole object, while passing by reference or const reference avoids the object copy. const reference is preferable when no modification is required.

Q2 – Return by Reference and Its Dangers

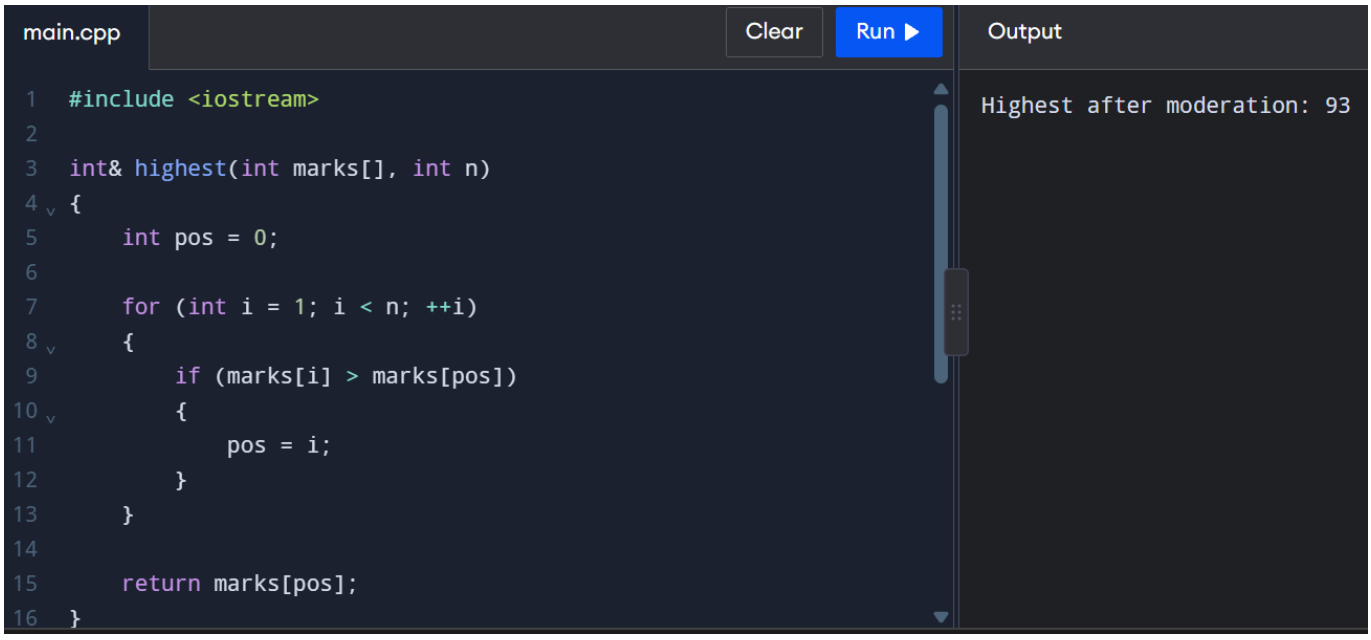
A reference returned by a function can refer directly to an existing array element. The caller can then modify that element, even by assigning to the function call. Returning a reference to a local variable is invalid because the local object is destroyed when the function returns; using the returned reference is undefined behaviour. A compiler normally warns about returning a reference to a local variable.

```
#include <iostream>

int& highest(int marks[], int n)
{
```

```
    int pos = 0;
    for (int i = 1; i < n; ++i)
    {
        if (marks[i] > marks[pos])
        {
            pos = i;
        }
    }
    return marks[pos];
}
int main()
{
    int marks[] = {72, 91, 84, 88};
    highest(marks, 4) += 2;
    std::cout << "Highest after moderation: "
              << highest(marks, 4) << '\n';
    return 0;
}
```

Expected result:



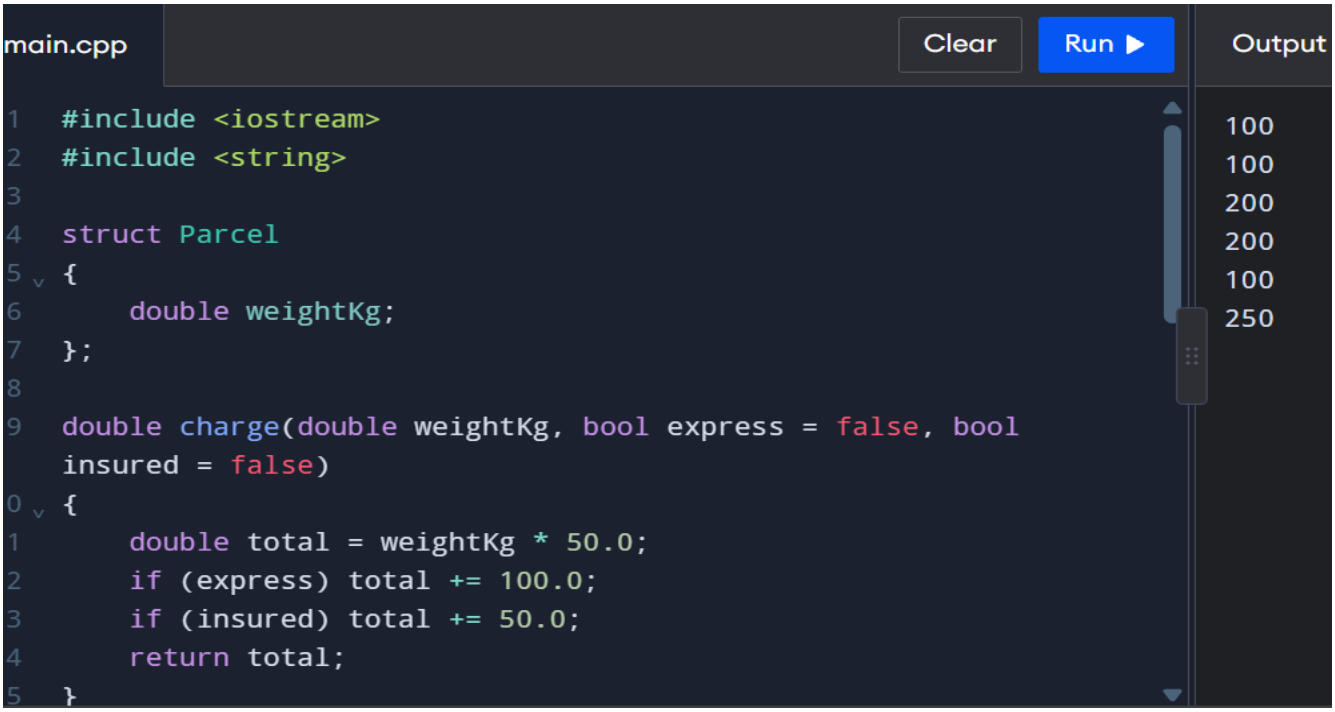
Q3 – Default Arguments and Function Overloading (6 Marks)

The original set of charge(double weight), charge(double weight, bool express) and charge(double weight, bool express=false, bool insured=false) creates ambiguity because the first two functions and the third function can all be viable for some calls.

Unambiguous Design

```
#include <iostream>
#include <string>
struct Parcel
{
   double weightKg;
};

double charge(double weightKg, bool express = false, bool insured = false)
{
   double total = weightKg * 50.0;
   if (express) total += 100.0;
   if (insured) total += 50.0;
   return total;
}
double charge(int weightGrams, bool express = false)
{
   double weightKg = weightGrams / 1000.0;
   return charge(weightKg, express, false);
}
double charge(const Parcel& p, bool express = false, bool insured = false)
{
   return charge(p.weightKg, express, insured);
}
int main()
{
   std::cout << charge(2.0) << '\n';          // double overload
   std::cout << charge(2000) << '\n';         // int overload
   std::cout << charge(2.0, true) << '\n';    // double overload
   std::cout << charge(2000, true) << '\n';   // int overload
   std::cout << charge(2.0f) << '\n';         // promoted to double
   Parcel p{2.0};
   std::cout << charge(p, true, true) << '\n';
   return 0;
}
```



Prediction of Overload Selection

- charge(2) selects the int overload by exact match.
- charge(2.0) selects the double overload by exact match.
- charge(2.0f) selects the double overload through floating-point promotion.
- charge(2000, true) selects the int overload by exact match.
- charge(2.0, true) selects the double overload by exact match.
- charge(p, true, true) selects the Parcel overload by exact match.

Overload resolution prefers an exact match over a promotion, and a promotion over a general conversion. Default arguments are considered after selecting a viable overload; they do not create separate overloads. Default arguments must be supplied from the right, because otherwise a later omitted parameter could not be distinguished reliably.

Alternative: a single Parcel structure with optional fields can reduce overload count. Overload resolution happens at compile time and adds no run-time cost. Passing a large Parcel by const reference avoids copying it.

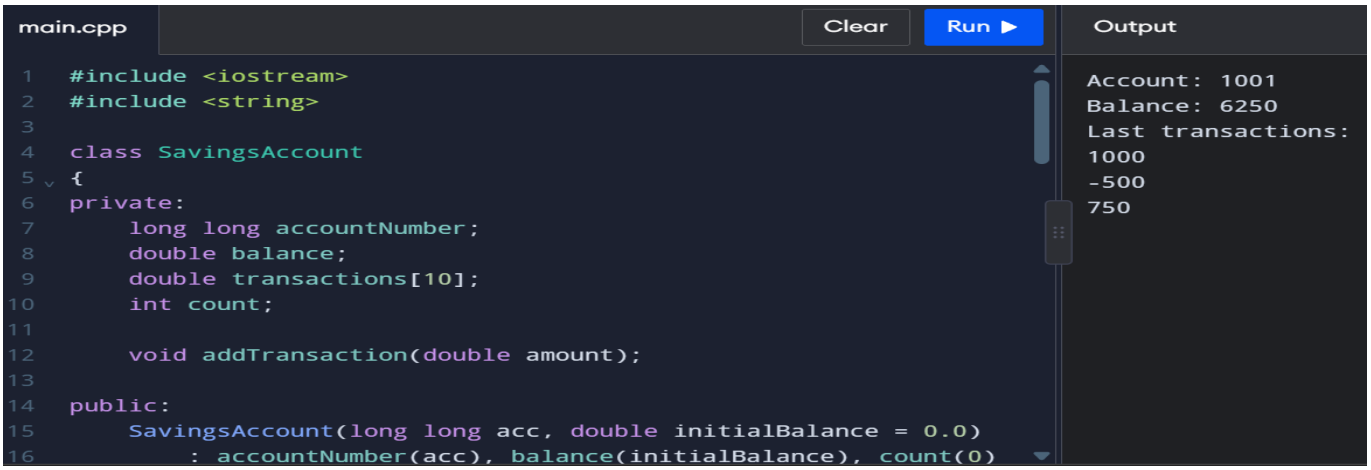
Q4 – Specifying a Class with an Array Inside It

The SavingsAccount keeps the account number, balance, ten transaction records and a count private. Public functions provide controlled access.

```
#include <iostream>
#include <string>
class SavingsAccount
{
private:
```

```
    long long accountNumber;
    double balance;
    double transactions[10];
    int count;
    void addTransaction(double amount);
public:
    SavingsAccount(long long acc, double initialBalance = 0.0)
        : accountNumber(acc), balance(initialBalance), count(0) {}
    void deposit(double amount)
    {
        if (amount <= 0) return;
        balance += amount;
        addTransaction(amount);
    }
    void withdraw(double amount);
    void miniStatement() const
    {
        std::cout << "Account: " << accountNumber << '\n';
        std::cout << "Balance: " << balance << '\n';
        std::cout << "Last transactions:\n";
        for (int i = 0; i < count; ++i)
            std::cout << transactions[i] << '\n';
    }
};
void SavingsAccount::addTransaction(double amount)
{
    if (count < 10)
    {
        transactions[count++] = amount;
    }
    else
    {
        for (int i = 1; i < 10; ++i)
            transactions[i - 1] = transactions[i];
        transactions[9] = amount;
    }
}
void SavingsAccount::withdraw(double amount)
{
    if (amount <= 0 || amount > balance) return;
    balance -= amount;
    addTransaction(-amount);
}
int main()
{
    SavingsAccount acc(1001, 5000);
    acc.deposit(1000);
```

```
    acc.withdraw(500);
    acc.deposit(750);
    acc.miniStatement();
    return 0;
}
```



Design Explanation

- The transaction array must be private so outside code cannot directly corrupt the account's history or count.
- With shifting, the 11th transaction moves transactions 1–9 one position left and places the new transaction at index 9. Insertion is O(10), generally O(k) for k retained records.
- A circular buffer avoids shifting and inserts in O(1), but requires a start/next index and slightly more logic for printing in chronological order.
- If the bank changes the requirement from 10 to 50, the fixed-array design only needs the capacity and related loops changed. A named constant is preferable so the class has one source of truth.

For the alternative std::deque design, push_back and pop_front are convenient and avoid manually shifting the stored records. A circular fixed array gives O(1) insertion with predictable memory. The fixed array uses O(k) fixed storage; deque also stores O(k) records but may have some implementation overhead.

Q5 – Static Members and Arrays of Objects (7 Marks)

A static data member belongs to the class rather than to an individual BusTicket object. Therefore the number of issued seats is stored once and shared by all objects.

Prediction for Token Program

Token::next starts at 100. Constructing counter[3] calls the constructor three times, producing IDs 101, 102 and 103. Constructing extra calls it once more, producing 104. Therefore the output is:

```
101 103 104 4
```

The static member is initialized outside the class definition. A static member function has no this pointer, so it can directly use only static members (unless an object is explicitly supplied).

Code

```
#include <iostream>
#include <string>
class BusTicket
{
   static int issued;
   static const int CAPACITY = 3;
   int seat;
   std::string name;
public:
   explicit BusTicket(const std::string& n) : seat(0), name(n)
   {
      if (issued < CAPACITY)
      {
         ++issued;
         seat = issued;
      }
   }
   static int count()
   {
      return issued;
   }
   void print() const
   {
      std::cout << name << ":"
            << (seat ? "Seat " + std::to_string(seat)
                  : std::string("Waiting"))
            << '\n';
   }
};
int BusTicket::issued = 0;
int main()
{
   BusTicket trip[4] = {
      BusTicket("Asha"),
      BusTicket("Bala"),
      BusTicket("Chitra"),
      BusTicket("Dev")
   };
   for (const BusTicket& t : trip)
      t.print();
   std::cout << "Issued: " << BusTicket::count() << '\n';
   return 0;
}
```

Expected output:



If tickets are created inside a function and that function is called twice, the static count is not reset between calls. It continues to represent the total seats issued by that class's shared counter. A global variable also uses one counter and O(1) time, but weakens encapsulation. A TicketOffice object can keep a separate counter for each bus, which is better when two buses must operate independently.

Q6 – Friend Functions and Inline Functions (8 Marks)

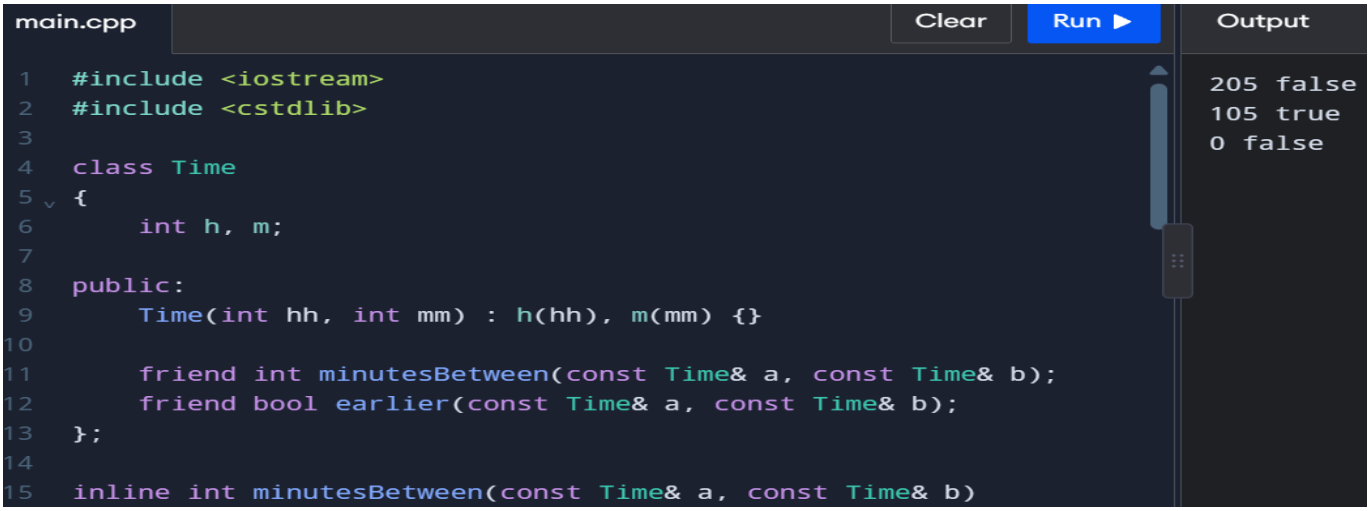
A friend function is suitable because minutesBetween(a,b) and earlier(a,b) treat two Time objects symmetrically. The function is not a member of either object, but friendship grants it access to private h and m. A member function would normally be called as a.minutesBetween(b) and can access a's private members directly.

Code

```
#include <iostream>
#include <cstdlib>
class Time
{
   int h, m;
public:
   Time(int hh, int mm) : h(hh), m(mm) {}
   friend int minutesBetween(const Time& a, const Time& b);
   friend bool earlier(const Time& a, const Time& b);
};
inline int minutesBetween(const Time& a, const Time& b)
{
   int aMinutes = a.h * 60 + a.m;
   int bMinutes = b.h * 60 + b.m;
   return std::abs(aMinutes - bMinutes);
}
inline bool earlier(const Time& a, const Time& b)
{
   int aMinutes = a.h * 60 + a.m;
   int bMinutes = b.h * 60 + b.m;
```

```
    return aMinutes < bMinutes;
}
int main()
{
    Time departure(9, 45), arrival(13, 10);
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha
              << earlier(arrival, departure) << '\n';
    Time t1(11, 30), t2(13, 15), same(11, 30);
    std::cout << minutesBetween(t1, t2) << ' '
              << earlier(t1, t2) << '\n';
    std::cout << minutesBetween(t1, same) << ' '
              << earlier(t1, same) << '\n';
    return 0;
}
```

For the required first test, the expected output is:



The two later tests verify a pair on either side of noon and equal times. The equal-time difference is 0 and earlier(t1,same) is false.

Inline Explanation and Complexity

- inline is a request to the compiler to permit expansion of a small function at the call site; it does not guarantee physical inlining.
- Both functions perform a constant number of arithmetic operations, so time complexity is O(1) and extra space is O(1).
- With -O0, the compiler may retain function calls, so inline may have little or no measurable benefit. With -O2, the compiler can inline automatically and optimize arithmetic, often making the difference small.
- Making h and m public would remove the need for friendship but would weaken encapsulation because any code could modify invalid times directly.

Deployment decision: keep the friend functions for this two-object, read-only comparison because they preserve private data and express the operation naturally. Member functions are also valid if the design strongly prefers object-oriented receiver syntax. Public data members are the least desirable option.

Q7 – Diagnose, Redesign and Validate

Selected program: Q4 SavingsAccount transaction history. This is a strong redesign candidate because the original fixed array shifts every stored transaction when the 11th and later transactions arrive.

A. Diagnose the Limitation

The original design stores the latest 10 transactions in an array. When it is full, adding a transaction shifts nine elements left before inserting the new one. The shift is O(k), where k is the number of retained transactions. As the required history grows from 10 to 50 or more, each insertion can require many assignments. The design also contains manual index-management code.

B. Design Objective

Objective: keep exactly the most recent 50 transactions while making insertion O(1) per transaction, without changing the public deposit(), withdraw() and miniStatement() interface.

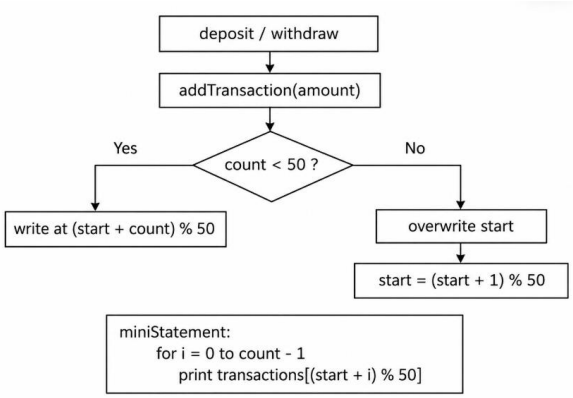
C. Revised Solution

Use a circular buffer. Keep a fixed array of 50 values plus a start index and count. When the buffer is not full, append at (start + count) % CAPACITY. When full, overwrite the oldest element at start and advance start. This uses O(50) storage and O(1) insertion. The trade-off is slightly more complicated printing logic.

```
class SavingsAccount
{
private:
   static const int CAPACITY = 50;
   long long accountNumber;
   double balance;
   double transactions[CAPACITY];
   int start;
   int count;
   void addTransaction(double amount)
   {
      int index = (start + count) % CAPACITY;
      if (count < CAPACITY)
      {
         transactions[index] = amount;
         ++count;
      }
      else
      {
```

```
            transactions[start] = amount;
            start = (start + 1) % CAPACITY;
        }
    }
public:
    SavingsAccount(long long acc, double initial)
        : accountNumber(acc), balance(initial), start(0), count(0) {}
    void deposit(double amount)
    {
        if (amount > 0)
        {
            balance += amount;
            addTransaction(amount);
        }
    }
    void withdraw(double amount)
    {
        if (amount > 0 && amount <= balance)
        {
            balance -= amount;
            addTransaction(-amount);
        }
    }
    void miniStatement() const
    {
        std::cout << "Account: " << accountNumber << '\n';
        std::cout << "Balance: " << balance << '\n';
        for (int i = 0; i < count; ++i)
        {
            int index = (start + i) % CAPACITY;
            std::cout << transactions[index] << '\n';
        }
    }
};
```

Class Design / Pseudocode



D. Predict Before Implementing

Prediction: insertion will change from O(k) shifting to O(1) index updates. With 50 retained records and a large number of deposits/withdrawals, the redesigned version should perform fewer element assignments per transaction.

E. Implement and Validate

Use the same sequence of transactions for both versions, first with fewer than 50 entries and then with more than 50. Verify that both versions print the same latest 50 transactions in the same order. Measure a large number of insertions using std::chrono.

```
#include <chrono>
#include <iostream>

int main()
{
   const int N = 1000000;
   auto start = std::chrono::high_resolution_clock::now();
   // Run the original shifting implementation here.
   // Run N deposits/withdrawals.
   auto end = std::chrono::high_resolution_clock::now();
   std::chrono::duration<double> elapsed = end - start;
   std::cout << "Elapsed time: " << elapsed.count() << " seconds\n";
   return 0;
}
```

main.cpp Clear Run ▶ Output

```cpp
#include <chrono>
#include <iostream>

int main()
{
    const int N = 1000000;

    auto start = std::chrono::high_resolution_clock::now();

    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double> elapsed = end - start;
    std::cout << "Elapsed time: " << elapsed.count() << "
seconds\n";
```

```
Elapsed time: 1.3e-07 seconds
```

Do not invent timing values in the final report. Record the actual values printed by your compiler and place them in a table such as: Original = ___ seconds; Circular buffer = ___ seconds; Number of transactions = 1,000,000.

F. Defend the Final Design

The circular buffer achieves the objective because each new transaction requires only a few index operations instead of shifting all retained records. It uses the same asymptotic storage, O(k), but gives O(1) insertion. The original fixed array is still preferable when the history is very small, the simplest possible implementation is required, and insertion frequency is low. The redesigned solution is preferable when the history is larger or transaction volume is high.

Quick Complexity Summary

Question	Main operation	Time	Extra space
Q1	Fee concession	O(1)	O(1)
Q2	Find highest mark	O(n)	O(1)
Q3	Charge calculation	O(1) run time	O(1) for scalar inputs
Q4	Add transaction, shifting	O(k)	O(k) fixed array
Q5	Create ticket / count	O(1)	O(1) shared counter
Q6	Compare two times	O(1)	O(1)
Q7	Circular-buffer insertion	O(1)	O(k) fixed storage